

Motif Search in GraphEasy — the Theory

Backtracking, spanning orders, and symmetry breaking: how the implementation relates to the textbook algorithms

GraphEasy (p1GraphEasy-con-AutoTuner)

September 2026

Abstract

This is the theory companion to `MOTIF_SEARCH_EXPLAINED.tex`. It answers one question in depth: *what algorithm is actually running when you write `a->b; b->c;` and ask for the count?* The short answer is a depth-first backtracking search over the pattern’s variables — pick a vertex for `a`, then a vertex for `b`, check the required edges, recurse — and the longer answer is how the pattern’s *spanning structure* controls the enumeration, how symmetry breaking turns counting into “one representative per orbit”, what the complexity is, and how this compares to VF2, RI, nauty-style canonical labelling and ESU enumeration. Two verified experiments anchor the theory: a $K_{2,2}$ bi-fan orbit of size 4 reported as exactly one match, and the effect of variable order on the search plan.

Contents

I	The mental model, checked against the code	1
1	“Loop over <code>a</code> , then loop over <code>b</code> , check the edge, recurse”	1
2	Where a spanning tree of the pattern enters — and where it does not	2
3	The two backends, in this language	2
II	The problem, formally	2
4	Definitions	3
5	Counting modulo symmetry	3
III	The search	3
6	Why depth-first backtracking is the right skeleton here	3
7	Complexity	4
8	The canonicity reduction (proof of the fast form)	4
9	Ordering: what a spanning tree does and does not buy	5
IV	Relation to the classical algorithms	5

V	Correctness obligations and the naming trap	6
10	Obligations	6
11	The naming trap (verified)	6
A	Annotated search trees	7
B	Formula card	7

Part I

The mental model, checked against the code

1 “Loop over a, then loop over b, check the edge, recurse”

The description in the question is essentially the algorithm. Concretely, for a pattern with variables a, b, c :

1. **Level 0 (variable a):** loop over candidate vertices.
2. **Level 1 (variable b):** for each candidate, check the constraints that involve only **a** and **b** — for $a \rightarrow b$ that means $\text{kind}(b, a)$ and $\text{kind}(a, b)$ must equal the pattern’s requirements (both directions, so that “no edge” is also a check).
3. **Level 2 (variable c):** same, against **a** and **b**;
4. **Recurse to depth k :** a complete assignment is a match; report it (or count it).

That skeleton is exactly what `runtime.c:1271-1315` implements (`motif_collect_backtrack_runtime`) and what the emitted IR nest implements (`MotifIRBuilder.cpp:246-293`). The one place where the two backends differ is *where the candidates come from*, and that is where the “spanning tree” in the question comes in.

2 Where a spanning tree of the pattern enters — and where it does not

Think of the pattern as a graph whose vertices are the variables $\{a, b, c, \dots\}$ and whose edges are the motif edges (plus, implicitly, the “must be absent” pairs, which are also constraints, just with the requirement 0). Two roles:

- **Tree edges drive enumeration.** Once some variable is bound, a candidate for a variable adjacent to it in the pattern can be taken from that variable’s adjacency list — one CSR row — instead of scanning all n vertices. A connected order of the variables therefore gives every level after the first a row to walk. In the implementation, this is the `LevelPlan` (`MotifPattern.h`): `source = OutRow` with a `driverVar` (“candidate is an out-neighbour of $A[\text{prev}]$ ”), or `InRow` for the mirror case, or `FullScan` when no tree edge is available.
- **Non-tree (chord) edges are check-only constraints.** This is the standard pattern of subgraph-search algorithms: enumerate along a spanning tree, verify the remaining edges as constraints. In the implementation every pair (j, prev) appears in the level’s `checks` list with *both* directions; the driving edge is not removed from the list, because the row proves the edge *exists* but not that its *kind* (weight sign) matches.

What the implementation does *not* do: it does not compute a minimum spanning tree, nor any optimal order. The variable order is simply the order in which the variables first appear in the motif edges (source before target, `ASTBuilder.cpp:230-243`), and the driver for each level is chosen by a recency heuristic: the most recently bound variable that has a required edge to the current level (`MotifPattern.cpp:127-168`). Section 5 discusses what an optimal order would be and why we can afford not to compute it.

3 The two backends, in this language

	runtime backend (default)	IR backend (<code>GRAPHEASY_MOTIF_BACKEND=ir</code>)
candidate source	always $0..n-1$	<code>FullScan</code> , or one CSR row per level (<code>GRAPHEASY_MOTIF_ADJDRIVE=1</code>)
consistency test	required matrix built at run time	constants baked in, same test, both directions
injectivity	<code>used[n]</code> array	k pairwise compares
canonicity	$O(k!)$ search per complete assignment	lower-bound compares, eval- uated while descending
output	bindings (so for each motif works)	count only, today

So the mental model “loop over all vertices, then loop over b” describes the runtime backend verbatim, and describes the IR backend once adjacency driving is enabled — with the improvement that each level after the first walks a row rather than the whole vertex set.

Part II

The problem, formally

4 Definitions

Let $G = (V, E)$ be a directed graph with a *kind function*

$$\kappa_G(u, v) = \begin{cases} 0 & \text{no edge } u \rightarrow v, \\ 1 & \text{edge with positive weight, or an unweighted edge,} \\ -1 & \text{edge with negative weight,} \\ 2 & \text{edge with weight 0.} \end{cases}$$

A *pattern* P has variables $X = \{x_0, \dots, x_{k-1}\}$ and a total requirement matrix

$$R : X \times X \rightarrow \{0, +1, -1\}, \quad R(i, j) = \begin{cases} +1 & \text{if the motif wrote } x_i \rightarrow x_j, \\ -1 & \text{if the motif wrote } x_i - | x_j, \\ 0 & \text{otherwise (induced: the edge must be absent).} \end{cases}$$

An *embedding* is an injective map $\sigma : X \rightarrow V$ such that for all $i \neq j$,

$$\kappa_G(\sigma(i), \sigma(j)) = R(i, j) \quad (\text{and, when } R(i, j) = +1 \text{ or } -1, \text{ the first-hit edge weight has that sign}).$$

The fourth line of the definition of κ_G is worth reading twice: a zero-weight edge satisfies *neither* $+1$ nor -1 nor 0 — it poisons the pair in both directions. This was verified experimentally (MOTIF_SEARCH_EXPLAINED.tex, §Stage 7): with a zero-weight chord, all three candidate patterns return 0).

5 Counting modulo symmetry

The number that a motif query reports is *not* the number of injections σ satisfying the constraints. Let

$$\text{Aut}(P) = \{\pi \in S_k : R(\pi(i), \pi(j)) = R(i, j) \ \forall i, j\}$$

be the automorphism group of the pattern. Two embeddings σ and $\sigma \circ \pi$ with $\pi \in \text{Aut}(P)$ use the same vertex set with permuted variable names; a motif count reports one representative per orbit.

This is not pedantry — it is what makes “ $K_{2,2}$ contains one bi-fan” true even though there are four injective bi-fan embeddings of a four-vertex bipartite graph. Verified experiment (pattern `a->x; a->y; b->x; b->y`; on the $K_{2,2}$ graph $0, 1 \rightarrow 2, 3$): `numMotifs` = 1, and the reported binding was the canonical one.

Part III

The search

6 Why depth-first backtracking is the right skeleton here

The constraints of R are *binary* and *monotone with respect to the binding order*: the constraint between two variables is fully decided the moment the later of the two is bound. Therefore a partial assignment that fails a constraint can never be extended to a full embedding — pruning is safe, and no backtracking beyond the offending level is needed. This is the entire justification for the “check b; if b fails, next b” structure.

The alternative enumeration strategy — generate all k -vertex induced subgraphs and classify each (ESU/Wernicke, Kavosh, MODA) — is the right choice when you want *all* motifs of size k at once. For a single fixed pattern, direct backtracking examines far fewer partial states, because the pattern’s edges are used as filters from the first level on.

7 Complexity

Naively the search tree has n branches per level and depth k , so $O(n^k)$ leaves; each leaf costs $O(k^2)$ to check and $O(k!)$ to canonicalise in the reference scheme. Three reductions, all implemented:

1. **consistency pruning**: a level only expands candidates that satisfy all pairs to earlier variables; the branching factor at level j is the number of vertices admissible given the first j choices;
2. **adjacency driving**: with tree edges, the candidate set at level j is one CSR row, so its size is the degree, not n ; for sparse graphs this is the difference between $O(n^k)$ and $O(\prod_j d_j)$;
3. **symmetry breaking**: the canonical lower bounds reject a candidate subtree as soon as it would produce a non-representative embedding, so the $|\text{Aut}(P)|$ -fold overcounting is removed *during* the descent, not by a post-filter at the leaves.

Worst case remains exponential in k (subgraph isomorphism is NP-hard), but $k \leq 6$ is enforced by the semantic analyser (`SemanticAnalyzer.cpp:987`), which is exactly the regime where a pattern-specialised nest with constants and k nested loops is practical.

8 The canonicity reduction (proof of the fast form)

Reference test (what the runtime does): an assignment A is canonical iff no automorphism image $A \circ \pi$ is lexicographically smaller than A , where the comparison walks $i = 0, 1, \dots, k-1$.

Fast test (what the emitter does): for each non-identity $\pi \in \text{Aut}(P)$, let $p = \min\{i : \pi(i) \neq i\}$; require $A[p] < A[\pi(p)]$.

Claim. For injective A , the two tests agree.

Proof. For a given π , the permuted vector $B = A \circ \pi$ satisfies $B[i] = A[\pi(i)]$. Since A is injective and π is a bijection, $B[i] \neq A[i]$ exactly when $\pi(i) \neq i$; hence the first coordinate where B can differ from A is p . Lexicographic comparison therefore reduces to the single comparison at p . Moreover $\pi(p) > p$: if $\pi(p) = q < p$ then, because p is the first moved index, $\pi(q) = q$; but then $\pi(q) = q = \pi(p)$ would contradict injectivity of π . So $A[p] < A[\pi(p)]$ is a lower bound on the coordinate p — which is bound *before* $\pi(p)$ in the search — and can be evaluated while descending.

□

Worked instance (verified). The bi-fan pattern $\mathbf{a} \rightarrow \mathbf{x}; \mathbf{a} \rightarrow \mathbf{y}; \mathbf{b} \rightarrow \mathbf{x}; \mathbf{b} \rightarrow \mathbf{y}$; has $|\text{Aut}(P)| = 4$: the sources a, b are interchangeable, the targets x, y are interchangeable, independently. The bound-producing automorphisms are π_{ab} (first moved index a), π_{xy} (first moved index x) and their product; the emitted nest therefore carries exactly two lower bounds:

$$A[a] < A[b], \quad A[x] < A[y].$$

On the $K_{2,2}$ graph this drops the four embeddings to the single canonical binding — measured: `numMotifs` = 1.

The FFL pattern $\mathbf{a} \rightarrow \mathbf{b}; \mathbf{a} \rightarrow \mathbf{c}; \mathbf{b} \rightarrow \mathbf{c}$; has trivial automorphism group (the directed triangle with a bypass has no symmetry: swapping b, c would have to map the present edge $b \rightarrow c$ to the absent edge $c \rightarrow b$), so no lower bounds are emitted and both embeddings of the five-arc example are reported — measured: 2, with bindings $(0, 1, 2)$ and $(0, 1, 3)$.

9 Ordering: what a spanning tree does and does not buy

- A *connected* variable order guarantees every level after the first has a row to walk (`OutRow/InRow`); a disconnected pattern forces a `FullScan` level, and — under induced semantics — all of that level’s checks are “must be non-adjacent” constraints, which is expensive but correct.
- The *choice of driver* within a level matters for cache behaviour (pick the most recently bound variable) and for the branching factor (pick a low-degree variable). The implementation picks by recency; VF2-style solvers additionally order variables by degree. At $k \leq 6$ one could brute-force all $k! \leq 720$ orders at compile time and keep the cheapest under a static cost model — a natural, cheap future improvement, not a requirement today.
- The pattern’s *written* order matters for the plan: the variables are indexed in first-appearance order, so rewriting the motif edges changes the level plans (and speed). It does not change the answer — verified: the FFL written as $\mathbf{b} \rightarrow \mathbf{c}; \mathbf{a} \rightarrow \mathbf{b}; \mathbf{a} \rightarrow \mathbf{c}$; still counts 2.

- A minimum spanning tree is not the right objective even in theory: for enumeration cost we would minimise the *product of candidate-set sizes* (a treewidth-flavoured objective), not edge weights, and the pattern’s edges carry no weights to minimise.

Part IV

Relation to the classical algorithms

algorithm	idea	relation to this implementation
plain backtracking	assign variables in order, check constraints	the runtime backend, verbatim; the correctness reference
VF2 / VF2++	feasibility sets, look-ahead, adaptive ordering	stronger pruning and variable ordering; not implemented here — at $k \leq 6$ the bookkeeping is not repaid
RI (Bonnici et al.)	spanning tree + back-edge verification	exactly the shape of the adjacency-driven IR nest: tree edges enumerate, chords check
nauty / McKay canonical labelling	canonically label the whole graph to dedupe isomorphic subgraphs	we dedupe <i>pattern orbits</i> , not graph isomorphism classes; at $k \leq 6$ the orbit check is a handful of compares
ESU / Kavosh / MODA	enumerate all k -subgraphs, then classify	the better tool for “all motifs at once”; for one fixed pattern, direct matching wins

The differential evidence that the pattern-level theory is implemented consistently: on a 2000-vertex, 6000-arc graph the FFL count is 26 under the runtime backend, the IR nest, and the adjacency-driven nest — three code paths, one answer — with per-run times 0.279 s, 0.219 s and 0.016 s respectively (`MOTIF_SEARCH_EXPLAINED.tex`, §Measured behaviour).

Part V

Correctness obligations and the naming trap

10 Obligations

- **Soundness:** every reported/counted assignment satisfies all pair constraints in both directions, is injective, and is a canonical representative. Enforced by construction (the checks run before the recursion) and by the induced-edge test.
- **Completeness:** every canonical embedding is counted. The filters are *necessary conditions* only, so no valid assignment is discarded; pruning removes only partial assignments that already violate a constraint.
- **Termination:** finite domains, finite depth, no cycles in the recursion.
- **Agreement of the two canonicity tests:** proved in §8; the reference test is kept in `MotifPattern` for unit-testing against a transcription of the runtime.

- **Agreement of the probe:** the emitted `__ge_motif_edge_kind` transcribes the runtime’s `graph_directed_edge_kind_runtime`, including first-hit-wins and the zero-weight poison.

11 The naming trap (verified)

The `for each motif (...)` binding list is **positional**. The variables of a pattern are collected in *first-appearance order*, scanning edges left to right and source before target (`ASTBuilder.cpp:230-243`). Consequences:

- for `{ a->x; a->y; b->x; b->y; }` the internal order is (a, x, y, b) — so the correct binding spelling is `for each motif (a,x,y,b)`, not `(a,b,x,y)`;
- the semantic analyser only checks the *arity* (`SemanticAnalyzer.cpp:687`), so a wrong order silently permutes the names.

Demonstration. Pattern written as `{ b->c; a->b; a->c; }` (internal order b, c, a) on the five-arc graph, printing `hasEdge(G,a,b)` and `hasEdge(G,a,c)` per match, 2 matches:

binding spelling	printed pairs	meaning
<code>(a,b,c)</code> (wrong order)	1 0 / 1 0	“a” is really b , “c” is really a ; the second probe asks for an
<code>(b,c,a)</code> (first-appearance order)	1 1 / 1 1	both probes are pattern edges, as they should be

Practical rule: write the motif edges so that the variables appear in the same order as the `for each` list, or list them in first-appearance order. (A friendlier compiler would match names instead of positions, or reject a permutation; today it does neither.)

A Annotated search trees

FFL on the five-arc graph. Pattern `a->b; a->c; b->c;`, `Aut = 1`, no bounds. Depth-first with levels a, b, c :

```

a=0 accept (all pairs checked vacuously at level 0)
  b=1 kind(1,0)=0 ok, kind(0,1)=1 ok
    c=2 kind(2,0)=0 ok, kind(0,2)=1 ok, kind(2,1)=0 ok, kind(1,2)=1 ok -> MATCH (0,1,2)
    c=3 ... same shape -> MATCH (0,1,3)
    c=1,0 rejected by injectivity
  b=2 kind(2,0)=0 ok, kind(0,2)=1 ok
    c=3 kind(3,2)=0 ok, kind(2,3) must be 1 -> 0 -> reject (no 2->3)
  b=3 kind(3,0)=0 ok, kind(0,3)=1 ok
    c=2 kind(2,3) must be 1 -> 0 -> reject
a=1 ...
a=2 ...

```

Bi-fan on $K_{2,2}$: the orbit of size four becomes one. Pattern `a->x; a->y; b->x; b->y;`, bounds $a < b, x < y$. The four embeddings $(a, b, x, y) \in \{(0, 1, 2, 3), (1, 0, 2, 3), (0, 1, 3, 2), (1, 0, 3, 2)\}$ all satisfy the edge constraints; the lower bounds accept only $(0, 1, 2, 3)$. Measured count: 1.

B Formula card

quantity	expression
kind function	$\kappa_G(u, v) = 0$ (no edge), 1 (unweighted or $w > 0$), -1 ($w < 0$), 2 ($w = 0$)
embedding condition	$\kappa_G(\sigma(i), \sigma(j)) = R(i, j)$ for all $i \neq j$, σ injective
automorphism group	$\text{Aut}(P) = \{\pi : R(\pi(i), \pi(j)) = R(i, j)\}$
canonical representative	σ such that no $\sigma \circ \pi$ is lexicographically smaller
fast bounds	for each non-identity π : $A[p] < A[\pi(p)]$, $p = \min\{i : \pi(i) \neq i\}$
brute-force cost	$O(n^k)$ leaves, $O(k^2)$ checks per leaf; row-driven levels replace n by the degree